

Modern Cryptography

Discrete logarithms and RSA

Discrete logarithms and RSA

- Discrete Logarithms
- DL and CDH Games
- Finding Cyclic Groups
- RSA

Discrete Logarithms

Discrete Logarithms

Discrete Logarithms

- Logarithm (로그):
 - 일반적인 수학에서 로그는 지수화(Exponentiation) 연산을 거꾸로 되돌리는 역산 함수이다.
- Discrete (이산적):
 - 암호학에서는 실수 범위가 아닌, 유한한 개수의 원소만 존재하는 '군(Group)' 내부에서만 연산이 이루어진다.
 - 숫자들이 소수 p 를 법(mod)으로 하여 불연속적으로 끊어져 존재하기 때문에 '이산(Discrete)'이라는 수식어가 붙는다.

Discrete Logarithms

- 암호학적 중요성:
 - 실수 체계의 로그와 달리, 이산적인 군 내부에서는 지수 연산이 매우 빠르지만 그 역산인 이산 로그를 구하는 것은 극도로 어렵다.
 - 이러한 '일방향성(One-wayness)'이 현대 암호학적 안전성의 기반이 된다.

Discrete Logarithms

Mathematical Definition of DLP

Mathematical Definition of DLP

- Background

- 특정 원소 g 를 거듭제곱하여 전체 공간을 빠짐없이 생성할 수 있을 때, 그 g 를 Generator라고 정의하며 이러한 군을 Cyclic Group이라 한다.
- 이산 로그는 이 Cyclic Group의 특성을 역으로 이용한 대수학적 함수이다.

Mathematical Definition of DLP

- Definition

- Order가 m 인 순환 군 $G = \langle g \rangle$ 와 군의 생성자 $g \in G$ 가 존재한다고 가정.
- 이때 군 G 에 속하는 임의의 원소 a 에 대하여, 다음 방정식을 만족하는 고유한 지수 $i \in Z_m$ 이 반드시 단 하나 존재한다.

$$g^i = a$$

- 이 수식을 만족하는 지수 i 를 'Base(밑) g 에 대한 a 의 Discrete Logarithm(이산 로그)'라고 정의하며, 다음과 같이 표기한다.

$$i = \text{DLog}_{G,g}(a)$$

Mathematical Definition of DLP

- 대수학적으로 이산 로그 함수는 지수화 함수의 완벽한 역함수로 작동한다.
- 따라서 다음 두가지의 항등식이 항상 성립한다.
 - $\text{DLog}_{G,g}(g^i) = i$ for all $i \in \mathbb{Z}_m$
 - $g^{\text{DLog}_{G,g}(a)} = a$ for all $a \in G$

Mathematical Definition of DLP

- Ex: 모듈러 11 환경에서의 이산 로그 역산 전개:

i	0	1	2	3	4	5	6	7	8	9
$2^i \bmod 11$	1	2	4	8	5	10	9	7	3	6

a	1	2	3	4	5	6	7	8	9	10
$\text{DLog}_{G,2}(a)$	0	1	8	2	4	9	7	3	6	5

Mathematical Definition of DLP

- 역산 계산:

- $a = 1$ 일 때: $2^0 = 1 \equiv 1 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(1) = 0$
- $a = 2$ 일 때: $2^1 = 2 \equiv 2 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(2) = 1$
- $a = 4$ 일 때: $2^2 = 4 \equiv 4 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(4) = 2$
- $a = 8$ 일 때: $2^3 = 8 \equiv 8 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(8) = 3$
- $a = 5$ 일 때: $2^4 = 16 = (11 \times 1) + 5 \equiv 5 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(5) = 4$
- $a = 10$ 일 때: $2^5 = 32 = (11 \times 2) + 10 \equiv 10 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(10) = 5$
- $a = 9$ 일 때: $2^6 = 64 = (11 \times 5) + 9 \equiv 9 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(9) = 6$
- $a = 7$ 일 때: $2^7 = 128 = (11 \times 11) + 7 \equiv 7 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(7) = 7$
- $a = 3$ 일 때: $2^8 = 256 = (11 \times 23) + 3 \equiv 3 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(3) = 8$
- $a = 6$ 일 때: $2^9 = 512 = (11 \times 46) + 6 \equiv 6 \pmod{11}$ 이므로, $\text{DLog}_{G,2}(6) = 9$

Mathematical Definition of DLP

- 이러한 모듈러 지수 연산을 수행하면 결과값 $a \in \{1, 2, \dots, 10\}$ 전체가 지수 $i \in \{0, 1, \dots, 9\}$ 와 중복 없이 완벽하게 1:1로 대응됨을 확인할 수 있다.
- 하지만 숫자가 커지면, a 만 보고 i 를 찾는 것은 불가능에 가깝다.

Mathematical Definition of DLP

- 위수가 m 인 순환 군 $G = \langle g \rangle$ 와 생성자 $g \in G$ 가 주어졌을 때, 임의의 원소 $X \in G$ 에 대한 이산 로그 값 $x = \text{DLog}_{G,g}(X)$ 를 찾는 가장 기초적인 알고리즘은 Brute-force(전수 조사)이다.
- 이 알고리즘은 $g^x = X$ 를 만족하는 지수 x 를 찾기 위해, 탐색 공간인 0부터 $m - 1$ 까지의 모든 정수를 순차적으로 대입한다.

Mathematical Definition of DLP

Input: X in G

Desired Output: $x = \text{DLog}_{\{G, g\}}(X)$

for $x = 0, \dots, m - 1$ do

 if $g^x = X$ then

 return x

Mathematical Definition of DLP

- 탐색 공간 내에 반드시 정답이 존재하므로 수학적으로 항상 올바른 해를 반환한다.
- 하지만 루프를 1회 수행할 때마다 지수화 연산이 요구되며, 최악의 경우 군의 위수 m 번만큼 연산을 반복해야 하므로 시간 복잡도는 $O(m)$ 이 된다.
- 암호학에서 입력값의 크기는 정수 자체가 아니라 이진수 인코딩의 비트 길이인 $|p|$ 로 측정되므로, 이 복잡도는 $O(2^{|p|})$ 이 된다.

Discrete Logarithms

Computing DLog: Z_p^* vs EC_p

Computing DLog: Z_p^* vs EC_p

- Finding Cyclic Groups
 - Fact1: Let p be a prime. Then Z_p^* is cyclic.
 - Fact2: Let G be any group whose order $m = |G|$ is a prime number. Then G is cyclic.
- $|Z_p^*| = p - 1$
- p 가 2보다 큰 홀수 소수일 때, $p - 1$ 은 항상 짝수이므로 소수가 될 수 없다.
- 따라서 군의 위수가 소수임을 전제로 하는 Fact2는 Fact1을 증명하거나 함의하지 않으며, 두 정리는 독립적으로 작용한다.

Computing DLog: Z_p^* vs EC_p

- Z_p^* vs EC_p

Group	Time to find discrete logarithms
Z_p^*	$e^{1.92(\ln p)^{1/3}(\ln \ln p)^{2/3}}$ (roughly) subexponential time
EC_p	$\sqrt{p} = e^{\ln(p)/2}$ exponential time

- 주로 유한체의 곱셈군인 Z_p^* 와 유한체 상의 곡선군 EC_p 가 주로 사용된다.
 - Z_p^* 의 복잡도: 대략 $e^{1.92(\ln p)^{1/3}(\ln \ln p)^{2/3}}$ 으로 계산된다.
 - EC_p 의 복잡도: $\sqrt{p} = e^{\ln(p)/2}$ 이다.

Computing DLog: Z_p^* vs EC_p

- Ex: In Z_p^*

$ p $ in bits	When
431	2005
530	2007
596	2014
768	2016
795	2019

- Z_p^* : 2019년 기준으로 약 795비트 크기의 소수 p 에 대한 이산 로그 계산이 기록 되었다.
- EC_p : 현재 기록은 약 114비트 수준이다.

Computing DLog: Z_p^* vs EC_p

- Ex: In Z_p^*

결론: 비트 길이가 114비트로 동일할 때, Z_p^* 는 무력화되지만 EC_p 는 해독 한계 수준의 방어 장벽을 유지한다.

- Z_p^* : 2019년 기준으로 약 795비트 크기의 소수 p 에 대한 이산 로그 계산이 기록 되었다.
- EC_p : 현재 기록은 약 114비트 수준이다.

Discrete Logarithms

Why EC Groups & Moore's Law

Why EC Groups & Moore's Law

- 곱셈군 (Z_p^*)에는 준지수 시간 해독 알고리즘이 존재하여 보안 성능이 저하된다.
- 타원 곡선 군 (EC_p)은 기하학적 점 가산 연산을 채택하여 대수적 해독을 무력화하고, 최적 복잡도를 순수 지수 시간 $O(\sqrt{p})$ 에 묶어둔다.
- 이때 연산 능력이 지수적으로 증가한다는 무어의 법칙(18개월마다 2배) 하에서 두 군의 키 크기 확장 효율성은 극명한 격차를 보인다

Why EC Groups & Moore's Law

- Ex: 80-bit

- 1단계: 80-bit 보안을 위한 최소 비트 길이 (k) 계산

- Z_p^* (준지수 시간 복잡도 함수): $e^{1.92(\ln p)^{1/3}(\ln \ln p)^{2/3}} \approx 2^{80} \Rightarrow k = 1024$ bits

- EC_p (순수 지수 시간 복잡도 함수): $\sqrt{p} = 2^{80} \Rightarrow p = 2^{160} \Rightarrow k = 160$ bits

- 2단계: 세제곱 법칙(Cubic Cost) 기반의 연산 비용 비교

- 모듈러 연산 비용은 비트 길이 k 의 세제곱에 비례한다.

- 타원 곡선 군($k = 160$)의 연산 비용을 기본 단위 $T = 160^3$ 으로 정의할 때, 곱셈 군($k = 1024$)의 연산 비용 비율을 구한다.

$$\frac{1024^3}{160^3} = \left(\frac{1024}{160} \right)^3 = (6.4)^3 = 262.144 \approx 260$$

- 결론: 동일 보안 수준에서 타원 곡선 군이 곱셈군보다 약 260배 연산 효율이 높다.

Why EC Groups & Moore's Law

- Ex: 80-bit
 - 3단계: 무어의 법칙(t 년)에 따른 키 크기 확장 속도 비교
 - 공격자 연산력이 $2^{t/1.5}$ 배 증가할 때, EC_p 는 복잡도 매칭을 통해 선형적으로 증가한다. $\frac{k_{new}}{2} = \frac{k_{old}}{2} + \frac{t}{1.5} \Rightarrow k_{new} = k_{old} + 1.33t$
 - 결과 분석: 타원 곡선 군은 매년 약 1.33 비트의 선형 확장만으로 무어의 법칙을 방어하지만, 곱셈군 Z_p^* 는 준지수 함수의 한계로 인해 초선형적인 폭발적 키 크기 팽창(1024 $\rightarrow$ 2048 $\rightarrow$ 4096-bit)을 강제당한다.

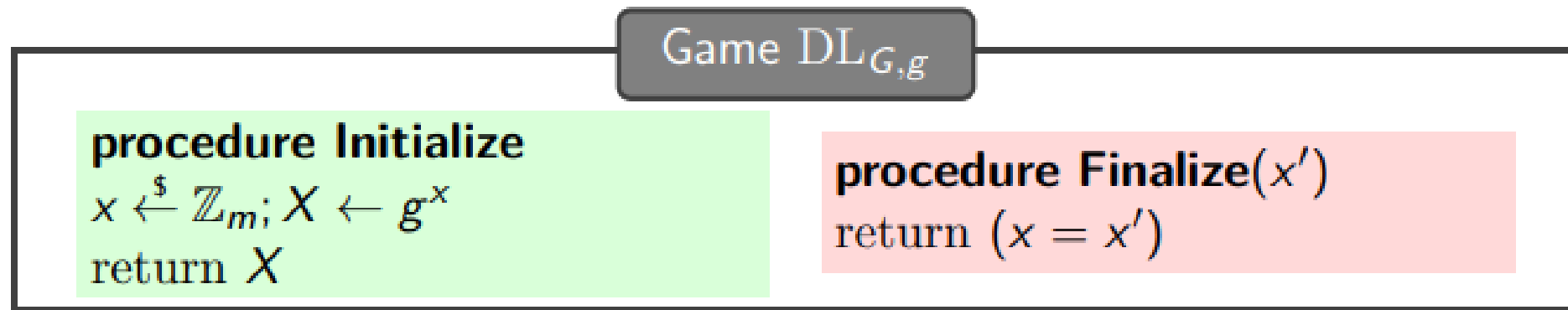
DL and CDH Games

DL Game

- 배경:
 - 이산 로그 문제의 난해성을 직관이 아닌 수학적 모델로 엄밀하게 평가하기 위함.
 - 위수가 m 인 순환 군 $G = \langle g \rangle$ 에서 공격자 A와 시스템 간의 상호작용을 게임 형태로 정의한다.

DL Game

Let $G = \langle g \rangle$ be a cyclic group of order m .



- Initialize (초기화): 시스템은 무작위 지수 $x \leftarrow \mathbb{Z}_m$ 를 무작위로 선택하고, $X \leftarrow g^x$ 를 계산하여 공격자에게 공격값 X 를 제공한다.
- Finalize (검증): 공격자는 이산 로그 역산을 시도하여 도출한 지수 x' 를 제출하며, 시스템은 $(x = x')$ 여부를 판별하여 결과를 반환한다.

DL Game

- DL-Advantage:
 - 공격자 A 가 이 게임에서 정확한 x 를 찾아내어 승리할 확률을 $Adv_{G,g}^{dl}(A) = Pr[DL_{G,g}^A \Rightarrow true]$ 로 정의한다.
 - 안전한 암호 시스템을 구축하려면 이 확률이 현실적으로 무시할 수 있을 만큼 극히 낮아야 한다.

CDH Problem & Relationship to DL

- CDH 문제: CDH(Computational Diffie-Hellman) 문제는 실제 Diffie-Hellman 비밀 키 교환 프로토콜의 안전성을 직접적으로 뒷받침하는 핵심 난제이다.
- 목표: 생성원 g 와 네트워크에 공개된 두 값 $X = g^x \in G, Y = g^y \in G$ 가 주어졌을 때, 공격자는 이 정보만을 가지고 두 지수가 결합된 최종 비밀 키 $g^{xy} \in G$ 를 계산해야 한다.

CDH Problem & Relationship to DL

- Relationship:
 - 만약 공격자가 이산 로그 문제를 풀 수 있는 알고리즘을 가졌다면, 공개값 X 에서 지수 $x \leftarrow DLog_{G,g}(X)$ 를 도출한 뒤 반대편 공개값에 지수화 (Y^x)를 수행하여 CDH 문제를 쉽게 해결할 수 있다.
 - 역으로 "CDH 문제 풀 수 있으면 DL 문제도 풀 수 있는가?"는 미해결 문제이다.
 - 일단 매우 어렵다고 간주.

CDH Game

Let $G = \langle g \rangle$ be a cyclic group of order m .

Game $\text{CDH}_{G,g}$

procedure Initialize

```
 $x, y \xleftarrow{\$} \mathbb{Z}_m$   
 $X \leftarrow g^x; Y \leftarrow g^y$   
return  $X, Y$ 
```

```
procedure Finalize( $Z$ )  
return ( $Z = g^{xy}$ )
```

- Initialize (초기화): 시스템은 두 개의 무작위 지수 $x, y \leftarrow \mathbb{Z}_m$ 를 선택하고, $X \leftarrow g^x$ 와 $Y \leftarrow g^y$ 를 계산하여 두 공개값을 공격자에게 제공한다.
- Finalize (검증): 공격자는 획득한 두 공개값을 바탕으로 도출한 최종 비밀 키 후보 Z 를 제출하며, 시스템은 이것이 실제 비밀 키와 동일한지 ($Z = g^{xy}$) 검증하여 결과를 반환한다.

CDH Game

- CDH-Advantage:
 - 공격자 A 가 CDH 게임에서 정확한 결합 비밀 키를 도출하여 승리할 확률은 $Adv_{G,g}^{cdh}(A) = Pr[CDH_{G,g}^A \Rightarrow true]$ 로 정의된다.

Finding Cyclic Groups

Finding Cyclic Groups

- 배경:
 - 암호학적 시스템이 안전하고 정상적으로 작동하기 위해서는 거대한 크기의 순환 군과 그 군의 전체 공간을 생성하는 생성자를 효율적으로 찾아 구축해야 한다.
 - 이를 위해 시스템은..
 - 적합한 형태의 거대한 소수 p 를 찾고,
 - Z_p^* 내에서 무작위 원소 g 를 뽑아 생성자인지 테스트하는 과정을 거친다.

Finding Cyclic Groups

- 소수 탐색 알고리즘 (Findprime(k)):

Alg Findprime(k)

do

$p \xleftarrow{\$} \{2^{k-1}, \dots, 2^k - 1\}$

until (p is prime and $(p - 1)/2$ is prime)

return p

- 단순한 첫 번째로 걸려온 소수 p 를 반환하는 것이 아니라, 안전 소수 조건을 통과할 때까지 난수를 뽑고 검증하는 필터링 엔진으로 작동한다.

Finding Cyclic Groups

- 안전 소수(Safe prime):
 - 단순히 p 가 소수인 것만 확인하는 것이 아니라, $q = (p - 1)/2$ 연산 결과인 q 역시 소수인지 동시에 검증한다.
 - 이처럼 $p = 2q + 1$ 구조를 만족하는 소수 p 를 암호학에서 안전 소수 (Safe prime)이라 부른다.

Finding Cyclic Groups

- 부분군 공격 (Subgroup attack):
 - 곱셈군 $\mathbb{Z}_p^*$ 의 전체 위수는 $p - 1$ 이다. 만약 일반적인 소수를 사용하면 $p - 1$ 이 2, 3, 5, 7 등 수많은 작은 소인수들을 약수로 가질 수 있다.

Finding Cyclic Groups

- 부분군 공격 (Subgroup attack):
 - 라그랑주 정리(Lagrange's Theorem)에 의해 군의 부분군 위수는 전체 위수의 약수여야 하므로, 일반 소수 환경에서는 원소의 개수가 아주 작은 소규모 부분군들이 무수히 많이 존재하게 된다.

Definition 3.1 유한한 군 G 의 원소의 개수를 군 G 의 차수(order)이라고 하며, $|G|$ 로 표기하도록 한다.

Theorem 3.1 (Lagrange Theorem) 유한한 군 G 의 부분군 H 에 대하여, $|H|$ 는 $|G|$ 의 약수이다.

Finding Cyclic Groups

- 부분군 공격 (Subgroup attack):
 - 악의적인 공격자가 Alice에게 정상적인 생성자 g 대신, 위수가 아주 작은(예: 위수가 3인) 부분군의 생성자 g_{small} 을 교환 값으로 슬쩍 밀어 넣는다고 가정해 보자.

Finding Cyclic Groups

- 부분군 공격 (Subgroup attack):
 - Alice가 계산하는 값은 $X_{small} = (g_{small})^x \pmod{p}$ 가 되는데, 이 값은 Alice의 비밀 키 x 가 아무리 거대하더라도 단 3가지 상태 $(g_{small}^0, g_{small}^1, g_{small}^2)$ 안에서만 맴돌게 된다.
 - 이를 통해 공격자는 거대한 비밀 키 x 를 직접 풀지 않고도, 각 작은 부분군에서의 나머지 정보 $x \pmod{3}$ 를 이용해 비밀 키 전체를 복원하여 부분군 공격을 수행할 수 있다.

Finding Cyclic Groups

- 안전 소수의 방어 원리:
 - Findprime 알고리즘이 필터링해 준 안전 소수($p = 2q + 1$)를 채택하면 전체 위수는 $p - 1 = 2q$ 가 된다.
 - 2와 q 는 모두 소수이므로, 이 군이 가질 수 있는 부분군의 위수는 오직 $\{1, 2, q, 2q\}$ 4가지로 제한된다.
 - 위수가 1이나 2인 무의미한 원소를 제외하면, 공격자가 악용할 수 있는 진부분군은 위수가 거대한 소수 q 인 부분군 딱 하나만 남게 된다.

Finding Cyclic Groups

- 안전 소수의 방어 원리:
 - 위수 q 는 수백~수천 비트의 거대한 소수이므로, 공격자가 부분군으로 탐색 범위를 좁히더라도 그 내부 공간($O(2^{|q|})$)이 우주적으로 넓어 전수 조사가 불가능해진다.
 - 즉, 안전 소수를 사용하는 것은 공격자가 침투할 수 있는 소규모 부분군들을 원천 봉쇄하는 방어 전략이다.

Finding Cyclic Groups

Primality testing

Primality testing

- 배경:
 - Findprime 알고리즘 루프 안에서 추출한 거대 정수 p 와 q 가 진짜 소수인지 판별해야 한다.
 - 일반적으로 생각할 수 있는 2부터 $\lceil \sqrt{N} \rceil$ 까지 모든 정수로 나누어떨어지는지 검사하는 알고리즘은 $O(\sqrt{N})$ 의 시간 복잡도를 가진다.
 - 정수의 크기가 수천 비트에 달하는 암호학적 환경에서 ($|N|$)에서 입력 비트 길이에 대한 지수 시간을 의미하므로 실제 시스템에서는 연산이 불가능하다.

Primality testing

- Polynomial time algorithms
 - 현대 암호학에선 이를 극복하기 위해 다항 시간 내에 구동되는 고속 알고리즘을 채택한다.
 - Miller-Rabin과 같은 $O(N \log^3 N)$ 복잡도의 확률론적 알고리즘을 사용하여 거대 정수의 소수 여부를 빠르게 판별한다.
 - 비록 $O(N \log^8 N)$ 수준의 결정론적 알고리즘도 존재하지만, 속도 측면에서 확률론적 알고리즘이 훨씬 우수하여 암호학적 소수 생성에 주로 쓰인다.

Finding Cyclic Groups

Density of primes

Density of primes

Let $\pi(N)$ be the number of primes in the range $1, \dots, N$. So if $p \xleftarrow{\$} \{1, \dots, N\}$ then

$$\Pr[p \text{ is a prime}] = \frac{\pi(N)}{N}$$

Fact: $\pi(N) \sim \frac{N}{\ln(N)}$

So

$$\Pr[p \text{ is a prime}] \sim \frac{1}{\ln(N)}$$

Density of primes

- Ex: $N = 2^{1024}$:
 - 실제 공개키 암호학에서 널리 쓰이는 1024비트 크기의 정수 공간을 가정해 보자.
 - 이때 무작위로 추출한 숫자가 소수일 확률은 $\text{약 } \frac{1}{\ln(2^{1024})} \approx 0.001488$, 즉 약 1/700의 확률을 가진다.

Density of primes

- Ex: $N = 2^{1024}$:

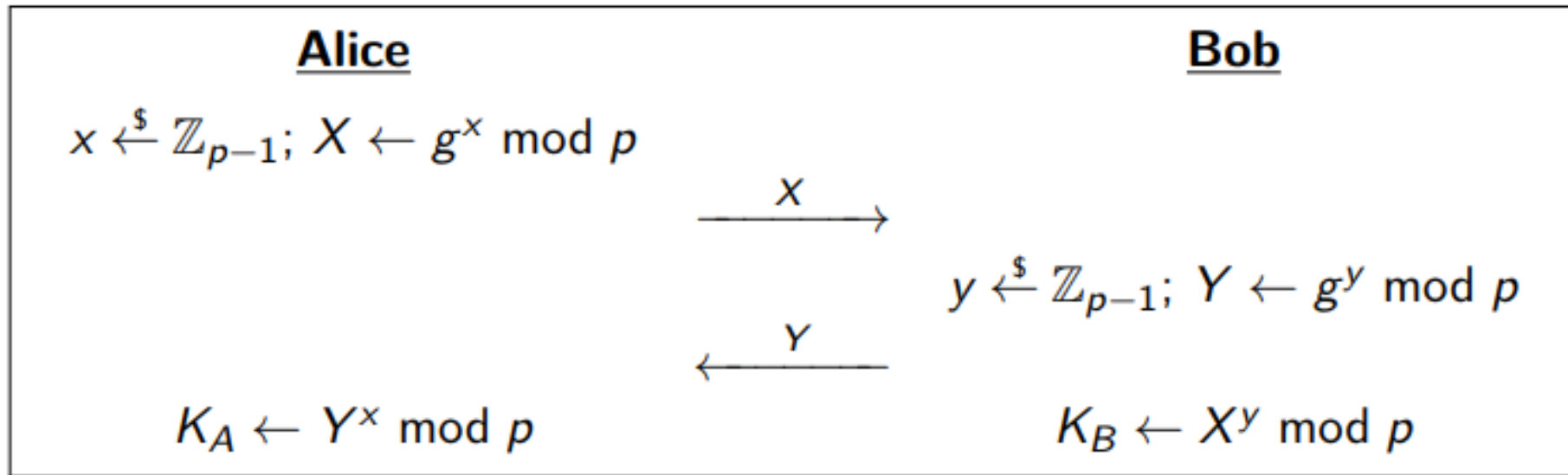
무작위 정수를 추출해 소수를 판별하는 루프는 평균적으로 대략 700번 이내의 반복만으로도 거대한 크기의 소수를 확정적으로 찾아낼 수 있다.

DH key exchange

- Diffie-Hellman key exchange 안전하게 성립하기 위해서는, 도청자가 풀 수 없는 거대한 소수 p 와 올바른 순환군의 생성자 g 가 파라미터로 주어져야 한다.

DH key exchange

- 전제 조건: 안전 소수 p 와 검증된 생성자 g 공개



- $Y^x = (g^y)^x = g^{xy} = (g^x)^y = X^y$ modulo p , so $K_A = K_B$
- Adversary is faced with the CDH problem.

DH key exchange

- 결론:
 - 보안성 측면 (부분군 공격 방어): 만약 Findprime 알고리즘이 안전 소수 ($p = 2q + 1$)를 엄선해 주지 않았다면, 공격자가 DH 교환 과정에서 소규모 부분군의 원소를 주입하여 Alice와 Bob의 비밀 지수 x, y 가 '부분군 공격'으로 프로토콜이 무력화되었을 것이다.

DH key exchange

- 결론:
 - 효율성 측면 (생성자 고속 공급): 프로토콜 가동을 위해 생성자 g 를 찾아야 할 때, 안전 소수 구조 덕분에 라그랑주 정리를 활용한 모듈러 지수 연산 검증만으로 올바른 g 를 빠르게 공급할 수 있게 되었다.

RSA

RSA math

RSA math

- 배경:
 - 이산 로그 기반 암호는 역산이 어려운 일방향성 함수에만 의존했다.
 - RSA는 구조를 아는 자(비밀키 소유자)만 통과할 수 있는 비밀 통로인 Trapdoor를 대수학적으로 설계하여 일방향성 치환을 구축한다.

RSA math

- Euler's Phi Function, $\varphi(N)$:
 - 1부터 N 까지의 정수 중 N 과 서로소인 정수의 개수를 나타내는 함수이다.
 - RSA는 두 개의 거대한 서로 다른 소수 p 와 q 를 택해 합성수 $N = p \cdot q$ 를 환경으로 삼는다.

RSA math

- Euler's Phi Function, $\varphi(N)$:

$$\varphi(N) = |Z_N^*| = (p-1)(q-1)$$

- 공격자에게 합성수 N 이 노출되더라도, N 을 구성하는 원래의 비밀 소수 p, q 를 모른다면 $\varphi(N)$ 값을 계산하는 것은 대수적으로 불가능하다.

RSA math

- Modulo inverse:
 - $\varphi(N)$ 과 서로소인 임의의 공개 지수(Public Exponent) e 를 선택한다.

$$\gcd(e, \varphi(N)) = 1$$

RSA math

- Modulo inverse:
 - EXD-GCD를 이용하면 다음 합동 관계를 만족하는 유일한 Private Exponent d 를 다항 시간 내에 찾아낼 수 있다.

$$ed \equiv 1 \pmod{\varphi(N)}$$

- 이때 임의의 메시지 $x \in \mathbb{Z}_N^*$ 에 대하여 다음 복원 공식이 항상 성립한다.

$$(x^e)^d \equiv x \pmod{N}$$

RSA math

- 증명:

- $ed \equiv 1 \pmod{\varphi(N)}$ 이므로, $ed = k \cdot \varphi(N) + 1$ 을 만족하는 정수 k 가 존재한다.

- 따라서 지수 법칙과 오일러 정리($x^{\varphi(N)} \equiv 1 \pmod{N}$)에 의해 다음과 같이 증명된다.

$$(x^e)^d \equiv x^{ed} \equiv x^{k \cdot \varphi(N) + 1} \equiv (x^{\varphi(N)})^k \cdot x^1 \equiv (1)^k \cdot x \equiv x \pmod{N}$$

RSA

RSA function and Trapdoor

RSA function and Trapdoor

- RSA 함수:
 - 모듈러스 N 과 암호화 지수 $e \in \mathbb{Z}_{\varphi(N)}^*$ 는 RSA 함수 $f: \mathbb{Z}_N^* \rightarrow \mathbb{Z}_N^*$ 를 다음과 같이 정의한다.

$$f(x) = x^e \bmod N \quad (\text{for all } x \in \mathbb{Z}_N^*)$$

RSA function and Trapdoor

- RSA 함수:
 - 복호화 지수(Decryption Exponent): $ed \bmod \varphi(N) = 1$ 을 만족하는 값 $d \in \mathbb{Z}_{\varphi(N)}^*$ 를 복호화 지수라고 부른다.
 - 치환 성질: f 는 군 $\mathbb{Z}_N^*$ 에서 자기 자신으로 매핑되는 1:1 대응 함수이다. 즉, 서로 다른 평문은 반드시 서로 다른 결과값으로 매핑된다.

RSA function and Trapdoor

- Trapdoor One-wayness:
 - 순방향 연산: (N, e) 가 주어지면, 고속 거듭제곱 알고리즘을 통해 누구나 다항 시간 내에 $f(x) = x^e \bmod N$ 을 쉽게 계산할 수 있다.
 - 역방향 연산: 오직 (N, e) 와 결과값 $y = f(x)$ 만 알고 있는 공격자가 원래의 입력값 x 를 역산하는 것은 일방향성에 의해 현실적인 시간 내에 불가능하다.

RSA function and Trapdoor

- Trapdoor One-wayness:
 - 하지만 비밀 정보인 복호화 지수 d 라는 Trapdoor 쥐고 있다면, 다음과 같이 단 한 번의 지수 연산으로 역함수 $f^{-1}(y)$ 를 순식간에 계산해 낼 수 있다.

$$f^{-1}(y) = y^d \bmod N = (x^e)^d \bmod N = x$$

RSA function and Trapdoor

- Ex:

Let $N = 15$. So

$$\mathbb{Z}_N^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$$

$$\varphi(N) = 8$$

$$\mathbb{Z}_{\varphi(N)}^* = \{1, 3, 5, 7\}$$

RSA function and Trapdoor

- Key Generation

- $p = 3, q = 5$

- $N = p \cdot q = 3 \cdot 5 = 15$

- $\varphi(15) = (3 - 1)(5 - 1) = 2 \cdot 4 = 8$

- $\mathbb{Z}_8^*$ 의 원소 중 $\gcd(e, 8) = 1$ 을 만족하는 $e = 3$ 채택.

- $3 \cdot d \equiv 1 \pmod{8}$ 를 만족하는 원소를 역산하면 $d = 3$ 도출.

- $(\because 3 \cdot 3 = 9 \equiv 1 \pmod{8})$

RSA function and Trapdoor

- 유한 곱셈군 $\mathbb{Z}_{15}^*$ 정의: $\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$
- RSA 치환 매핑 검증
 - 군의 모든 원소 x 에 대해 암호화($f(x) = x^3 \bmod 15$)와 복호화($f^{-1}(y) = y^3 \bmod 15$)를 수행하면, 다음과 같이 1:1 대칭 치환 구조가 성립한다.

RSA function and Trapdoor

- RSA 치환 매핑 검증
 - 군의 모든 원소 x 에 대해 암호화($f(x) = x^3 \bmod 15$)와 복호화($f^{-1}(y) = y^3 \bmod 15$)를 수행하면, 다음과 같이 1:1 대칭 치환 구조가 성립한다.

x	$f(x)$	$g(f(x))$
1	1	1
2	8	2
4	4	4
7	13	7
8	2	8
11	11	11
13	7	13
14	14	14

RSA function and Trapdoor

- 결론:

- 평문 $x = 2$ 는 순방향 함수에 의해 $f(2) = 8$ 로 치환되고, 복호화 지수 $d = 3$ 을 이용해 $f^{-1}(8) = 2$ 로 깔끔하게 복원된다.
- 평문 $x = 7$ 은 순방향 함수에 의해 $f(7) = 13$ 으로 치환되고, 복호화 지수 $d = 3$ 을 이용해 $f^{-1}(13) = 7$ 로 복원된다.
- 비밀 지수 d 를 모르는 공격자에게는 임의의 원소들이 무작위로 섞인 순열 테이블처럼 보이지만, 이를 알고 있는 수신자는 $y^d \bmod N$ 연산만으로 원래의 평문으로 귀환할 수 있음을 보여준다.